

Evaluation Comments Template

General Information

- Evaluator Name: _____ P5 _____
 - Application Name: _____ 7ep-demo _____
 - Refactoring Approach: _____ microrefact _____
-

1) Services Boundaries and Decompositions

1. Are all of the classes in the application mapped to their correct microservices based on their decomposition?

All classes have been well refactored, except (classes not found in microservices):

- PasswordResultEnums.java
- RegistrationStatusEnums.java
- LibraryActionResults.java
- Calculator.java

2. Are there classes that were not included in the decomposition but found in the monolith? How were they handled by the approach?

Describe any missing or unaccounted classes:

These classes were not part of decomposition clusters and were also not refactored into the microservices.

- AutoInsuranceScriptServer.java
- RegistrationUtilsTests.java
- AckermannStepDefs.java
- ParameterObjectTests.java
- CartesianProductStepDefs.java

The Refact app, according to the paper describing the approach, explicitly focuses on decomposing classes directly related to application business logic into independent microservices. Thus, non-core functionalities (such as test classes and script-handling utilities) are intentionally ignored,

3. Are there new classes added to the microservices? Do they still respect the single responsibility of these microservices?

Main.java was introduced by the refactoring approach to serve as the entry point for each microservice

Yes, the newly added class Main.java fully respects the Single Responsibility Principle within each microservice.

4. Has the dependency mapping between components been accurately preserved?

The dependency mapping between components has been mostly preserved, but with some shared classes duplicated across multiple microservices:

Classes shared across multiple microservices:

- StringUtils.java: found in microservices 0, 1, and 3
- ParameterObject.java: found in microservices 0, 1, and 3
- ServletUtils.java: found in all microservices (0, 1, 2, and 3)
- AssertionError.java: found in microservices 0 and 3
- IPersistenceLayer.java, SqlData.java, SqlRuntimeException.java, NotImplementedException.java, WebAppListener.java, PersistenceLayer.java, and EmptyDataSource.java: all found in microservices 0, 1, and 3
- CheckUtils.java: found in microservices 0 and 3

5. How well did the approach handle this aspect? (Score from 1 to 10)

6/10

6. Additional Comments

No comments

2) Correctness & Functional Equivalence:

1. Does the refactored application produce the same outcomes as the original monolith?

I've run some basic tests, and the results seem to be the same, but more tests need to be done with more cases.

2. Have any functional behaviors been inadvertently altered during the transformation?

Exist some differences typically include minor structural changes such as formatting, whitespace, comments, or declarations, rather than fundamental changes in logic.

3. Is the transformation strictly limited to modifying call mechanics, leaving the business logic unchanged?

Non, the transformation has been strictly limited to adjusting communication and modularity mechanics. The underlying business logic is preserved intact, maintaining functional equivalence between the original monolith and the refactored microservices.

4. Do the transformed interactions maintain the same logical flow as in the monolith?

What remains consistent:

- Business processes and sequences of operations (e.g., authentication → validation → response, library management → book registration → confirmation).
- Conditional logic (if-else branches, exception handling) and validation sequences remain intact.
- Endpoints and interactions are functionally equivalent (e.g., POST login requests still authenticate users correctly).

What has changed in interaction mechanics:

- Direct method calls between components within the monolith are replaced by RESTful inter-service communications between microservices.
- Data flow previously handled internally now occurs via serialized DTOs between services, potentially adding latency or changing synchronous calls to asynchronous or distributed interactions.

5. How well did the approach handle this aspect? (Score from 1 to 10)

7/10

6. Additional Comments

No comments

3) Code Transformation and Remote Invocations:

1. Are all relevant local method calls successfully replaced by their remote call counterparts?

Not all relevant local method calls have been converted into remote calls. Example: The expected REST or remote communication patterns described by the refactoring approach have not been fully applied in the analyzed classes (LoginServlet.java, RegisterServlet.java, LibraryRegisterBookServlet.java and others).

2. Are the request and response structures consistent with the original local calls?

Yes, currently consistent, but only because the transformation to remote calls was not actually performed.

If corrected to introduce remote calls, explicit verification of DTOs and REST interface correctness would be essential.

3. Are there any issues such as missing object mappings or incorrect handling of requests and responses?

There are no object has been mapped incorrectly.

4. Is data correctly serialized and deserialized during remote calls?

There are no immediate issues with missing object mappings or incorrect request/response handling in the existing refactored microservices because remote calls or DTO transformations have not yet been introduced.

Also, Method parameters and return types remain unchanged from the original monolith implementation, inherently avoiding mapping or transformation issues.

5. How well did the approach handle this aspect? (Score from 1 to 10)

6/10

6. Additional Comments

No comments

4) Data Handling and Consistency:

1. Is there an existing database in the monolith? How did the refactoring approach handle it?

Yes, the original monolithic application has an existing database. this existing database structure was preserved exactly without enhancement or adjustment for a truly distributed context.

2. How is a shared state managed or synchronized across the services?

No explicit shared state synchronization is present in the refactored microservices.

3. Is there any data duplication? Do you think that it is justified?

Each microservice maintains its own isolated instance of an embedded database, resulting in duplication of database schemas and potentially reference data.

Do you think that it is justified?

Yes, it simplifies initial development, allowing each microservice to run independently without immediate integration complexity.

4. Is the data still consistent? Were there any new consistency mechanisms added if needed?

Yes, data remains internally consistent within each individual microservice since each maintains its own isolated H2 in-memory database.

5. How well did the approach handle this aspect? (Score from 1 to 10)

7/10

6. Additional Comments

It seems to me that the database management has been very well refactored.

5) Performance Overhead:

1. Are there new mechanisms introduced that add overhead to existing interactions?

No new mechanisms such as explicit remote calls, REST-based inter-service communications, or distributed transactions have been introduced yet in the current refactoring. Also, each microservice still directly uses local method calls and isolated, embedded databases.

2. Do you think that this overhead is justified?

There's no overhead detected.

3. What mechanisms would you replace if you had to modify the code manually?

- Replace Local Calls with REST-based Remote Invocations.
- Replace Embedded Databases with Centralized or Shared Databases.
- Replace Direct Database Access with Proper DTOs and Data Mapping.
- Replace Synchronous Transactions with Asynchronous Event-driven Transactions.
- Replace Redundant Utility Class Duplication with Shared Library Modules

4. Are there any indications of over-segmentation leading to excessive inter-service communication?

No indication of excessive communication or over-segmentation.

5. How well did the approach handle this aspect? (Score from 1 to 10)

7/10

6. Additional Comments

No comment

6) Code Maintainability & Readability Post-Transformation:

1. Is the refactored code easy to understand and follow compared to the original monolith?

Non, both versions are generally easy to understand and follow. but the original monolith slightly edges out the microservice due to better inline documentation.

2. Has the transformation introduced excessive boilerplate code or complexity?

The refactoring has not introduced excessive boilerplate code because it has not fully implemented typical microservice patterns like DTOs or REST endpoints, which often lead to boilerplate.

3. How difficult is it to replace the mechanisms introduced by the approach if needed?

It easy to replace mechanisms due to minimal changes.

4. Does the approach include fault tolerance or error handling mechanisms?

No all seems working well.

5. How well did the approach handle this aspect? (Score from 1 to 10)

6/10

6. Additional Comments

No comment

7) Execution and Packaging:

1. Does the refactored version compile and/or package correctly?

Yes, it seems to be compiled fine.

2. Does the refactoring approach introduce new dependencies?

Yes, new dependencies like gRPC, Protocol Buffers, and leasing mechanisms are explicitly introduced.

With the presence of correct pom.xml files, these dependencies are explicitly defined and managed appropriately.

3. Does the approach introduce any new deployment automation mechanisms?

The refactoring approach has not explicitly introduced new external dependencies in the refactored microservices version. It uses the same existing classes, utilities, and embedded database (H2) that were already present in the original monolithic application.

4. How well did the approach handle this aspect? (Score from 1 to 10)

7/10

5. Additional Comments

It seems that everything is working well

8) Additional comments that do not fall into any aspect:

No comment